ProjetLocatif — Revue « boîte grise » des contrôles d’accès

Intervenant : Nicolas BRANDEJSKY (compte 8570) · **Date :** 2026-11-05 **Méthode :** analyse structurelle depuis un **accès légitime** (compte propre), par usage normal de l’API. **Aucune donnée de tiers consultée, aucune écriture. But :** cartographier la surface d’attaque des contrôles d’accès et préparer le test d’étanchéité (qui, lui, nécessite une recette + comptes A/B — voir §5).

> *Document confidentiel, destiné à l’éditeur de la plateforme (divulgation responsable).*

---

1. Architecture observée

- Stack **Laravel + Passport** (SPA). Auth = cookie de session `HttpOnly` + jeton Passport `laravel_token` (délivré automatiquement au chargement d’une page) + double-submit CSRF via `X-XSRF-TOKEN`.
- API REST sous `/api/*`, ~**348 routes** déclarées.

2. Signaux POSITIFS (le contrôle par rôle existe) ✅

Observé via des appels légitimes depuis le compte conseiller : - `POST /api/prospects/stats` → **403 « This action is unauthorized. »** : l’endpoint de stats est **refusé** au rôle conseiller. → un **contrôle d’autorisation par rôle** est bien en place. - `PUT /api/properties/{id}/delete` (suppression **définitive**) → **403** pour le rôle conseiller. → les actions sensibles/admin sont **gatées**. - CSRF effectif sur écriture, session `HttpOnly`, accès non authentifié refusé (cf. rapport de posture).

Conclusion partielle : il existe une couche d’autorisation **au niveau du rôle**. Reste à prouver l’autorisation **au niveau de l’objet** (point 3).

3. Surface de risque à vérifier (indices, non prouvés)

#	Indice	Pourquoi ça compte	Gravité

G1	**ID séquentiels** sur toutes les ressources (prospects ~`10xxx`, biens ~`11xxx`, ventes ~`17xx`)	Rend l'énumération triviale **si** l'autorisation n'est pas vérifiée par objet (BOLA/IDOR)	**potentielle** Élevée si non protégé
G2	**Objets très riches** (une fiche prospect expose situation financière, contacts secondaire	Impact RGPD fort en cas d'IDOR	Élevée (i

	s, documents)		mpact)
G3	**Table de routes complète** exposée côté client (Ziggy)	Reconnaissance facilitée	Faible/Moyenne
G4	**Permissions renvoyées au client** (`authorisations`)	Le serveur doit re-vérifier à chaque appel, pas se fier au client	À vérifier

G 5	Endpoints d'**export** (`/prospects/export`, `/sales/export`)	Vérifier qu'ils sont strictement limités au périmètre de l'appelant	À vérifier

4. Hypothèse centrale à trancher (BOLA / IDOR)

> *Un conseiller peut-il **lire ou modifier** une ressource (prospect / vente / bien) rattachée à **un autre** conseiller, en changeant l'`id` ?*

C'est **la** question de Robin. Elle **ne peut pas** être tranchée depuis le seul compte propre sans toucher aux données d'un tiers. À tester en recette (§5).

5. Protocole de test proposé (sans risque)

1. L'éditeur fournit une **recette** + **2 comptes de test** : A et B, chacun avec ses propres prospects/biens/ventes **fictifs**, qui **ne doivent pas se voir**.
2. Connecté en A, tenter de lire `GET /api/prospects/{idB}`, `…/sales/{idB}`, `…/properties/{idB}`.
   - **403/404** attendu = étanchéité OK.
   - **200** = IDOR en lecture → on constate sur **un seul** objet témoin, puis on arrête.
3. Idem en écriture : `POST /api/sales/{idB}` (modif), suppression — vérifier le refus.
4. Rejouer pour l'élévation de privilèges (endpoints réservés).
5. Rapport final avec fiches de constat (preuve minimale, données fictives uniquement).

6. Recommandations serveur (quel que soit le résultat)

1. **Policies Laravel par objet** : sur chaque `show/update/destroy`, vérifier que l'objet appartient au périmètre de l'utilisateur (pas seulement le rôle).
2. **API Resources filtrées** : ne renvoyer que les champs nécessaires selon le rôle.
3. **Tests d'autorisation automatisés en CI** (jeu de comptes A/B qui ne doivent pas

se voir).

4. **Réduire l’exposition** : table de routes, `authorisations`.
5. **Journalisation/alerte** sur l’énumération d’`id` et les pics de lecture.